

Низькорівневе програмування та програмування мікроконтролерів

З використанням мови програмування **Assembler**
Stack, Subroutines, Parameters

класрум **im3ozqq4**

Що таке стек?

Notes

Стек — це структура даних типу LIFO (last in, first out), яка підтримується апаратно у процесорах i386.

Він використовується для тимчасового збереження даних, параметрів виклику функцій та адрес повернення.

Стек є центральним механізмом у програмуванні на низькому рівні:
ВІН ДОЗВОЛЯЄ:

- викликати функції,
- передавати параметри,
- зберігати тимчасові дані
- забезпечувати рекурсивні виклики.

Структурування програм

Оператор **CALL** в асемблері відіграє ключову роль у структурованому програмуванні, оскільки дозволяє викликати підпрограми та забезпечує модульність коду.

Його використання дає змогу розділяти програму на логічні блоки, кожен з яких виконує окреме завдання.

Це значно підвищує читабельність і супроводжуваність програм, а також спрощує тестування та повторне використання коду.

Структурування програм

При виконанні **CALL** у стек зберігається адреса повернення, що дає змогу після завершення підпрограми виконати команду **RET** і повернутися до потрібного місця. Таким чином, **CALL** створює основу для реалізації функцій, рекурсії та організації програмної логіки.

Приклад

```
mov ax, 1  
mov bx, 2  
call add1  
...
```

```
add1: add ax, bx  
ret
```


Приклад

```
    mov     cx, n_rows
row:
    push    cx
    mov     cx, n_cols
col:
    add     al, [esi]
    inc     esi
    loop    col

    pop     cx
    loop    row
```

```
    xor     ax, ax
    mov     esi, table

    call    sum_table
    ...
```

```
sum_table:
    mov     cx, n_rows
rows:
    push    cx

    mov     cx, n_cols
cols:
    add     al, [esi]
    inc     esi
    loop    cols

    pop     cx
    loop    rows

    ret
```

Де тут стек?

Інструкція **CALL**
автоматично зберігає адресу
наступної інструкції (EIP) у стек
і здійснює перехід до функції

```
xor ax, ax
mov esi, table

call sum_table
...

sum_table:
    mov cx, n_rows
rows:    push cx

    mov cx, n_cols
cols:    add al, [esi]
         inc esi
         loop cols

    pop cx
    loop rows

ret
```


Де тут стек?

Інструкція **CALL**
автоматично зберігає адресу
наступної інструкції (EIP) у стек
і здійснює перехід до функції

```
xor ax, ax
mov esi, table

call sum_table
...

sum_table:
    mov cx, n_rows
rows:    push cx

    mov cx, n_cols
cols:    add al, [esi]
        inc esi
        loop cols

        pop cx
        loop rows

ret
```


004000b0	66 31c0	xor ax, ax
004000b3	bed5104000	mov esi, 0x4010d5
004000b8	e802000000	call 0x4000bf
004000bd	90	nop
004000be	90	nop
004000bf	66 b90300	mov cx, 3
004000c3	66 51	push cx
004000c5	66 b90400	mov cx, 4
004000c9	67 0206	add al, [esi]
004000cc	ffc6	inc esi
004000ce	e2f9	loop -7
004000d0	66 59	pop cx
004000d2	e2ef	loop -17
004000d4	c3	ret

sp : 0x4fffffffffed0

Stack memory									
00004fffffffffed0	01	00	00	00	00	00	00	00	00

sp : 0x4fffffffffec8

Stack memory									
00004fffffffffec8	bd	00	40	00	00	00	00	00	00
00004fffffffffed0	01	00	00	00	00	00	00	00	00

sp : 0x4fffffffffed0

Stack memory									
00004fffffffffed0	01	00	00	00	00	00	00	00	00

Семантично:

```
call sum_table
```

```
push $ + 1  
jmp sum_table
```

Але такого синтаксису нема, це для розуміння

Семантично:

```
ret
```

```
push ip  
jmp ip
```

Але такого синтаксису нема, це для розуміння

Передача параметрів

- Якщо параметри не влазять в реєстри,
- або параметрів більше ніж реєстрів

Передача параметрів

- Якщо параметри не влазять в реєстри,
- або параметрів більше ніж реєстрів

То для передачі параметрів можна використовувати стек

Приклад

```
004000b0  68230100...  push 0x123
004000b5  68340200...  push 0x234
004000ba  68560400...  push 0x456
004000bf  68670500...  push 0x567
004000c4  68780600...  push 0x678
004000c9  e80a0000...  call 0x4000d8
004000ce  90           nop
```

Stack memory

00004fffffffffea0	ce	00	40	00	00	00	00	00
00004fffffffffea8	78	06	00	00	00	00	00	00
00004fffffffffeb0	67	05	00	00	00	00	00	00
00004fffffffffeb8	56	04	00	00	00	00	00	00
00004fffffffffec0	34	02	00	00	00	00	00	00
00004fffffffffec8	23	01	00	00	00	00	00	00

Теоретично в середині функції можна використати ROP
Але тоді ми “загубимо” адресу куди повертатися після
виконання функції.

Приклад

```
004000b0  68230100...  push 0x123
004000b5  68340200...  push 0x234
004000ba  68560400...  push 0x456
004000bf  68670500...  push 0x567
004000c4  68780600...  push 0x678
004000c9  e80a0000...  call 0x4000d8
004000ce  90           nop
```

Stack memory

00004ffffffffffea0	ce	00	40	00	00	00	00	00
00004ffffffffffea8	78	06	00	00	00	00	00	00
00004ffffffffffeb0	67	05	00	00	00	00	00	00
00004ffffffffffeb8	56	04	00	00	00	00	00	00
00004ffffffffffec0	34	02	00	00	00	00	00	00
00004ffffffffffec8	23	01	00	00	00	00	00	00

Теоретично в середині функції можна використати ROP
Але тоді ми “загубимо” адресу куди повертатися після
виконання функції. Її потрібно якось зберігти...

Пролог / Епілог

Для доступу для параметрів, переданих через стек ми використовуємо наступний паттерн

```
push rbp
mov rbp, rsp

...

mov esp, ebp
pop rbp
ret
```

Пролог

Пролог функції зазвичай включає PUSH EBP та MOV EBP, ESP, щоб створити новий кадр для локальних змінних і параметрів

```
push rbp  
mov rbp, rsp
```


Епілог

Епілог функції відновлює старе значення ESP і EBP перед поверненням, забезпечуючи правильний вихід з кадру

```
mov rsp, rbp  
pop rbp  
ret
```

Доступ до параметрів

```
push rbp
mov rbp, rsp

mov rax, [rbp+8*2]
mov rbx, [rbp+8*3]
mov rcx, [rbp+8*4]
mov rdx, [rbp+8*5]
```

Stack memory								
00004fffffffffea0	00	00	00	00	00	00	00	00
00004fffffffffea8	c9	00	40	00	00	00	00	00
00004fffffffffeb0	67	05	00	00	00	00	00	00
00004fffffffffeb8	56	04	00	00	00	00	00	00
00004fffffffffec0	34	02	00	00	00	00	00	00
00004fffffffffec8	23	01	00	00	00	00	00	00

```
rax : 0x0567
rbx : 0x0456
rcx : 0x0234
rdx : 0x0123
```


Звільнення стеку від параметрів після виклику функції

...

```
mov rsp, rbp  
pop rbp  
ret
```

Stack memory

00004fffffffffeb0	67	05	00	00	00	00	00	00	00
00004fffffffffeb8	56	04	00	00	00	00	00	00	00
00004fffffffffec0	34	02	00	00	00	00	00	00	00
00004fffffffffec8	23	01	00	00	00	00	00	00	00

Звільнення стеку

Виглядає так, що після виклику функції нам потрібно почистити стек від параметрів, що нам вже не потрібні

Існує 2 стандарти:

cdecl

004000b0	6823010000	push 0x123
004000b5	6834020000	push 0x234
004000ba	6856040000	push 0x456
004000bf	6867050000	push 0x567
004000c4	e80e000000	call 0x4000d7
004000c9	48 83c420	add rsp, 0x20
004000cd	90	nop

Stack memory								
00004fffffffffed0	01	00	00	00	00	00	00	00

Stack memory								
00004fffffffffeb0	67	05	00	00	00	00	00	00
00004fffffffffeb8	56	04	00	00	00	00	00	00
00004fffffffffec0	34	02	00	00	00	00	00	00
00004fffffffffec8	23	01	00	00	00	00	00	00

Stack memory								
00004fffffffffea8	c9	00	40	00	00	00	00	00
00004fffffffffeb0	67	05	00	00	00	00	00	00
00004fffffffffeb8	56	04	00	00	00	00	00	00
00004fffffffffec0	34	02	00	00	00	00	00	00
00004fffffffffec8	23	01	00	00	00	00	00	00

Stack memory								
00004fffffffffeb0	67	05	00	00	00	00	00	00
00004fffffffffeb8	56	04	00	00	00	00	00	00
00004fffffffffec0	34	02	00	00	00	00	00	00
00004fffffffffec8	23	01	00	00	00	00	00	00

Stack memory								
00004fffffffffed0	01	00	00	00	00	00	00	00

stdcall

```
push 0x123
push 0x234
push 0x456
push 0x567

call sum_table
```

```
sum_table:
    ...
    ret 8*4
```

Stack memory								
00004fffffffffed0	01	00	00	00	00	00	00	00

Stack memory								
00004fffffffffeb0	67	05	00	00	00	00	00	00
00004fffffffffeb8	56	04	00	00	00	00	00	00
00004fffffffffec0	34	02	00	00	00	00	00	00
00004fffffffffec8	23	01	00	00	00	00	00	00

Stack memory								
00004fffffffffea8	c9	00	40	00	00	00	00	00
00004fffffffffeb0	67	05	00	00	00	00	00	00
00004fffffffffeb8	56	04	00	00	00	00	00	00
00004fffffffffec0	34	02	00	00	00	00	00	00
00004fffffffffec8	23	01	00	00	00	00	00	00

Stack memory								
00004fffffffffed0	01	00	00	00	00	00	00	00

Автоматизація генерації пролог / епілог

```
sum_table:  
    enter 8, 0  
    ...  
    leave  
  
    ret
```

8 - число байтів які резервуються під локальні змінні.
Автоматично додає код:

```
sub sp, 8
```

Висновки

Стек є фундаментальним елементом роботи процесора: він забезпечує функціональність викликів функцій, рекурсії та збереження контексту

Notes

Notes

Notes